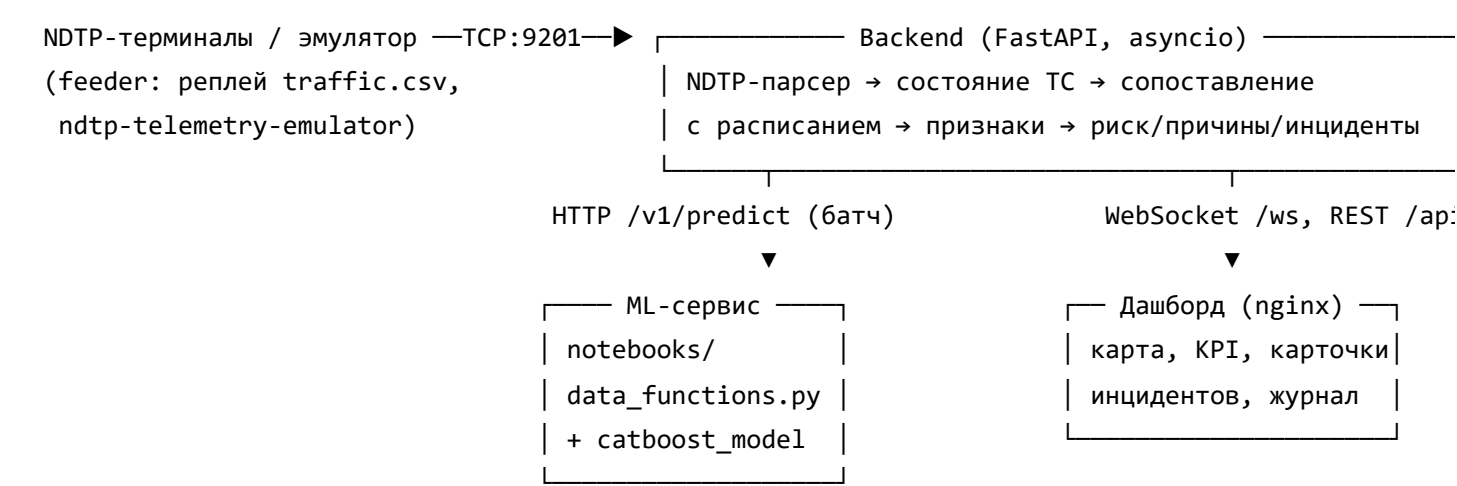


Предиктор задержек городского транспорта

Система в реальном времени принимает телеметрию ТС по протоколу **NDTP**, накладывает её на нитку эталонного расписания, с помощью ML-модели прогнозирует отклонение от графика **за 10–15 минут** до прибытия, определяет вероятную причину сбоя и выводит алерт с рекомендациями на дашборд диспетчера.



Запуск (одна команда)

Требуется Docker (Compose v2) и папка `dataset/` рядом с этим файлом (раздаётся отдельно, в git не хранится).

```
docker compose up -d --build
```

Что	Адрес
Дашборд диспетчера	http://localhost:8080
Swagger Backend	http://localhost:8000/docs
Swagger ML-сервиса	http://localhost:8001/docs

Что	Адрес
Управление реплеем (Swagger)	http://localhost:8090/docs
Приём NDTP (TCP)	localhost:9201

Поток стартует автоматически: реплей реальной телеметрии dataset/validate/traffic.csv с 07:30 МСК (утренний пик), ускорение $\times 10$. Через ~ 8 с после старта на дашборде появляются прогнозы.

Штатный эмулятор NDTP (опционально, параллельно с реплеем):

```
docker load -i dataset/ndtp-telemetry-emulator.tar
docker compose --profile emulator up -d      # конфиг: scripts/emulator-config.json → backend:9201
```

Эмулятор шлёт случайную навигацию около Москвы с unitId, которых нет в расписании, поэтому его ТС видны на карте как «без наряда» (серые). Он подтверждает совместимость парсера с протоколом, а сопоставимый с расписанием поток даёт feeder: это тот же NDTP-клиент (handshake, пауза 200 мс, realtime-пакеты, reconnect), но с координатами и временем из реальной телеметрии.

Остановка: `docker compose --profile emulator down`.

Модули

Сервис	Каталог	Роль
ml-service	services/ml_service	ML-ядро: инференс и обучение. HTTP-обёртка над notebooks/data_functions.py и models/catboost_model.cbm . ML-код не изменён: онлайн-запрос сериализуется в тот же табличный вид, что CSV датасета, и проходит через те же load_and_preprocess_* → build_dataset → predict . Эндпоинт /v1/train повторяет пайплайн ноутбука data_work.ipynb (выключен по умолчанию, ML_ENABLE_TRAIN=true).

Сервис	Каталог	Роль
backend	services/backend	API и оркестрация. TCP-сервер NDTP, сопоставление с расписанием, производные признаки, батч-вызовы ML, риск, причины, инциденты, деградация, метрики, REST, WebSocket.
feeder	services/feeder	Источник живого NDTP-потока: реплей <code>traffic.csv</code> плюс сценарии отказов (обрыв связи, мусор в канале).
dashboard	dashboard	BI-дашборд диспетчера (nginx, SPA на Leaflet; проксирует <code>/api</code> , <code>/ws</code> , <code>/feeder</code>).
libs/ndtp	libs/ndtp	NDTP-кодек без зависимостей; общий для backend и feeder.

Backend не зависит от готовности ML-сервиса: без него он работает в деградированном режиме.

Приём и парсинг NDTP

Реализован по `dataset/docs/Emulator-and-Telematic-Packets-Specification.md` и проверен на живом эмуляторе

(захваты трафика лежат в `libs/ndtp/tests/fixtures`).

- Кадр `NPL(15) + NPH(10) + тело` , little-endian. CRC-16/Modbus по NPH+телу, в NPL байты CRC переставлены.
- Handshake `NPH_SGC_CONN_REQUEST` и все пакеты с флагом `request` подтверждаются ответом `NPH_RESULT` .
- Ячейки: `Nav00` , `IntSensor02` , `Usi08` , `Can10` , `Lls15` , `Termo16` разбираются по полям. Для ячеек §6.7, бинарный layout которых в спецификации не описан, размер измерен на эмуляторе. Они пропускаются целиком, поэтому следующие ячейки пакета не теряются.
- Поточковый декодер собирает кадр из произвольных TCP-фрагментов. На мусоре, ложной сигнатуре `0x7E7E` или ошибке CRC он пересинхронизируется, не разрывая соединение (есть метрика `skipped_garbage_bytes`).
- `Nav00` → поля `traffic.csv` по таблице §8 README датасета: знаки широты/долготы из битов 5/6, валидность из бита 7, `peerAddress` → `unit_id` → `tr_id` .

- Отладочный `POST /api/ndtp/parse` разбирает hex-кадр прямо из Swagger (пример в Swagger — реальный кадр эмулятора).

Сопоставление с расписанием и признаки

- **Фактическое прибытие** определяется по геозоне 45 м. Проверяется не только точка GPS, но и отрезок трека между точками: GPS приходит раз в 10–15 с, это 60–80 м пути, и одиночная точка часто проскакивает остановку.
Время прибытия — момент ближайшего подхода (интерполяция).
Сопоставление идёт по порядку расписания в окне $[-8, +12]$ мин от плана. Так кольцевые маршруты (та же остановка через 20 мин) и остановки встречного направления не дают ложных срабатываний.
- **Текущее отклонение** `cur_dev_s` — отклонение на последней пройденной остановке, как в разметке.
- **Цель прогноза** — первая остановка с плановым временем в окне $(T+10, T+15]$ мин (как в разметке).
- **Производные признаки** (на дашборде и в диагностике): средняя скорость на сегменте и плановая скорость сегмента, время простоя, признак «стоит на остановке / вне остановки», возраст телеметрии, серия сбоев GPS, тренд отклонения по последним остановкам.

Риск, причины, инциденты

- **Уровень риска:** зелёный / жёлтый / красный. Жёлтый: прогноз опоздания > 120 с (порог `target_class=late`), опережение < -60 с или $P \geq 50\%$. Красный: прогноз ≥ 240 с, либо опоздание при $P \geq 75\%$. На карте цвет дублируется формой маркера (круг / треугольник / ромб) и подписью.
- **Вероятность опоздания** $P(\text{задержка} > 120 \text{ с})$ — калибровка поверх точечного прогноза: ошибка регрессора с `loss=MAE` аппроксимируется распределением Лапласа с масштабом, равным MAE модели на валидации (ML-сервис отдаёт его из `catboost_info`).

- **Причина и паттерн** — rule-based диагностика по признакам телеметрии: простой вне остановки (затор/ДТП/поломка), затянувшаяся стоянка, низкая скорость на участке, нарастание отставания, унаследованное опоздание, опережение, потеря связи, сбой GPS, пиковая нагрузка. Каждой причине соответствует рекомендация диспетчеру.
- **Инцидент** — эпизод жёлтого или красного риска по ТС. Он закрывается после 2 мин (время данных) в зелёной зоне. Диспетчер может «принять» инцидент.

Надёжность и деградация

Сбой	Поведение
Обрыв связи с источником телематики	Режим «Деградация» с баннером на дашборде. Часы данных продолжают идти с последним темпом потока, прогнозы строятся по последнему известному состоянию. После 30 мин без телеметрии модель работает на исторических признаках (<code>telemetry_missing=1</code> , статистики <code>sched_*</code>). ТС помечаются «данные устарели».
Реконнект	Восстанавливается автоматически, никаких действий не нужно. Терминал повторяет handshake.
Недоступен ML-сервис	Circuit breaker (3 ошибки → пауза 5 с). Прогноз переключается на baseline «= текущее отклонение», карточки помечаются тегом <code>baseline</code> . После восстановления ML возвращается сам.
Мусор / битые кадры	Пересинхронизация по сигнатуре, соединение не рвётся.
Перезапуск реплея (время назад)	Нитка графика и часы пересинхронизируются.
Медленный клиент WebSocket	Очередь снимков длиной 2 (хранится только свежий снимок), память не растёт.

Сбой	Поведение
Потеря связи браузера с Backend	Дашборд показывает последнее состояние, переключается на опрос REST и переподключается к WS.

Сценарии воспроизводятся из панели «Демо-управление потоком» на дашборде или через API фидера:
POST :8090/outage/60 , POST :8090/chaos/garbage , а также docker compose stop ml-service .

Измеренные характеристики

Условия: ноутбук, Docker Desktop (Windows). Сырые выводы лежат в scripts/out/ .

Производительность

Метрика	Значение
Холодный старт (образы собраны): backend / ML / первый прогноз / дашборд	6.2 с / 6.2 с / 7.4 с / 7.5 с
ML-инференс, батч 13 TC (живой поток), p50 / p95	18 мс / 20–24 мс
ML-инференс, батч 1 / 50 / 200 / 1000 точек (медиана)	29 / 20 / 47 / 82 мс (0.08 мс на точку)
Задержка «пакет → опубликованный прогноз», p50 / p95	~120 мс / ~470 мс
Цикл прогноза (все TC), p95	25–33 мс, перегрузок цикла 0
Приём NDTP: 1000 терминалов × 5 пакетов/с (в сети Docker)	~4970 пакетов/с без потерь , обработка пакета p95 23 мкс, API p50 1.5 мс

Задержка по телеметрии держится ниже 0.5 с при целевом ориентире 1–2 с. Очереди не копятя: пакеты, пришедшие за время цикла, коалесцируются в следующий батч (cycle_overruns=0 , потерь нет). Метрики доступны онлайн: GET /api/metrics , GET /metrics (Prometheus), панель «Производительность» на дашборде.

Качество (честная оценка онлайн-контура на labels_test.csv , 353 точки) — scripts/evaluate_online.py

Прогноз	MAE, с
Нулевой прогноз	103.3
Baseline cur_dev_s из разметки	93.4
Модель, cur_dev_s из разметки (офлайн-постановка)	71.1
Модель, cur_dev_s посчитан онлайн по GPS	92.2

Детектор прибытий (scripts/validate_matching.py) находит 78% плановых остановок; физический потолок около 80%, потому что в остальных местах нет GPS. Медиана ошибки времени прибытия относительно time_fact_begin — 7 с, 91% ошибок укладываются в 60 с.

Разница 71 → 92 с объясняется определением признака в разметке. Там cur_dev_s — факт на остановке с плановым временем ≤ T, а этот факт нередко наступает **после** T. Онлайн он ещё не известен, поэтому онлайн-контур использует последнее детектированное отклонение (вариант выбран по этой же оценке, см. CUR_DEV_MODE). Онлайн-точность также считается прямо в работе, по факту прибытия на целевую остановку: KPI «Точность онлайн» на дашборде.

Тесты и скрипты

```
# юнит-тесты (NDTP-кодек на реальных захватах эмулятора, сопоставление, риск, часы) — в образе t
docker run --rm -v "$PWD:/w" -w /w mos-transport/backend:1.0 sh -c "pip install -q pytest && py

python scripts/validate_matching.py           # качество сопоставления с расписанием (нужны pand
python scripts/evaluate_online.py            # MAE онлайн-контура и latency ML (стек поднят)
python scripts/load_test.py --units 1000 --rate 5 --duration 30  # нагрузка на приём NDTP
```

Основные настройки (env Backend)

Переменная	По умолчанию	Смысл
ML_URL	http://ml-service:8001	адрес ML-сервиса
PREDICT_INTERVAL_S / PREDICT_MIN_INTERVAL_S	1.0 / 0.15	максимальный и минимальный период цикла прогноза (событийный запуск)
STOP_RADIUS_M	45	радиус геозоны остановки
MAX_EARLY_S / MAX_LATE_S	480 / 720	окно сопоставления прибытия с планом
CUR_DEV_MODE	last_detected	оценка cur_dev_s до прибытия на остановку (last_detected / lower_bound)
LATE_THRESHOLD_S / RED_THRESHOLD_S	120 / 240	пороги жёлтого и красного уровня
LINK_TIMEOUT_S	15	через сколько секунд без пакетов включается деградация
DISPLAY_TZ_OFFSET_H	3	часовой пояс отображения (МСК)

Фидер: REPLAY_START (UTC), REPLAY_SPEED , REPLAY_LOOP , REPLAY_UNITS .

Что требуется от ML-части

ML-ядро в этой работе не менялось. Backend использует модель как есть, а перечисленное ниже закрыто

временными решениями, которые стоит заменить результатами ML-команды:

1. **Классификатор причины/паттерна сбоя.** Сейчас причина определяется rule-based диагностикой в Backend (services/backend/app/risk.py). Нужна модель (или разметка паттернов), которая по признакам

телеметрии и расписания выдаёт тип сбоя. Её выход встраивается в `diagnose()` как ещё одна причина с весом.

2. **Вероятность отклонения.** Модель — регрессор; вероятность получается аппроксимацией Лапласа по валидационной MAE.

Лучше отдавать откалиброванную вероятность напрямую: классификатор late/ontime/early или квантильные прогнозы

(`loss_function=Quantile / MultiQuantile`). ML-сервис вернёт их полем в `/v1/predict` .

3. **Устойчивость к `cur_dev_s` , посчитанному онлайн.** Обучение на `cur_dev_s` из разметки (там есть знание факта после T) даёт разрыв 71 → 92 с MAE онлайн. Рекомендуется обучать и валидировать на онлайн-версии признака (её генерирует `scripts/validate_matching.py` → `scripts/out/online_cur_dev_*.csv`) или добавить признаки позиции ТС относительно остановки.

4. **Поведение без телеметрии.** При `telemetry_missing=1` модель почти полностью «списывает» текущее отклонение.
Пример: `cur_dev_s=274` → прогноз 12 с. В режиме деградации это занижает риск. Стоит добавить в обучение примеры с пропуском телеметрии или ограничить дельту в этом режиме на стороне модели.

5. **Чувствительность к порядку пакетов с одинаковым `event_time` . `speed_diff` зависит от порядка строк**
при совпадающих метках времени. Офлайн это порядок строк CSV, онлайн — порядок поступления.
Паритет train/serve подтверждён для 351 из 353 тестовых точек, расхождения только в таких случаях.

Стоит сделать признак детерминированным (например, агрегировать дубликаты).

6. **Масштаб:** `tr_id` — категориальный признак, поэтому для новых ТС модель опирается только на остальные признаки. Для сети в тысячи ТС нужен признак маршрута или участка вместо ID.